DO NOW — RECALL

Lesson 141 · ACID Transactions, Locking and Redundanc

Name: _____ Class: _____ Date: _____

1. What does foreign-key referential integrity prevent?

2. Which is faster in the worst case on sorted arrays, linear or binary search?

3. Which logic gate outputs 1 only when both inputs are 1?

4. What is the accumulator for?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

A child key pointing to a nonexistent parent.

2. Model answer

Binary search: O(log n) rather than O(n).

3. Model answer

AND.

4. Model answer

Holding a CPU's working data/ALU results.

By the end, you can…

• Move £20: subtract account A and add account B.

• Both updates commit together or both roll back.

• Isolation/locks reduce interference; durable commits survive failures.

Vocabulary — know these terms

transaction — Group of DB operations treated as one unit of work.

ACID — Atomicity, Consistency, Isolation, Durability.

record locking — Restricting conflicting access while data is being changed.

redundancy — Duplicated resources/data for resilience, distinct from backup.

NEW LEARNING

Read, annotate and explain

Begin transaction

Move £20: subtract account A and add account B.

Protect consistency

Both updates commit together or both roll back.

Coordinate concurrent work

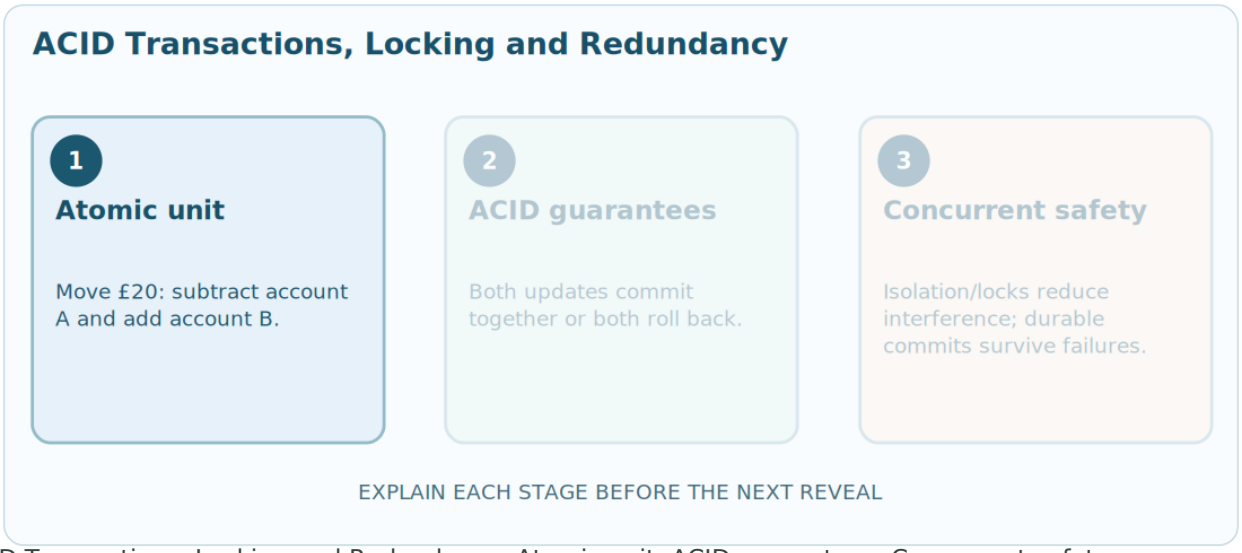
Isolation/locks reduce interference; durable commits survive failures.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

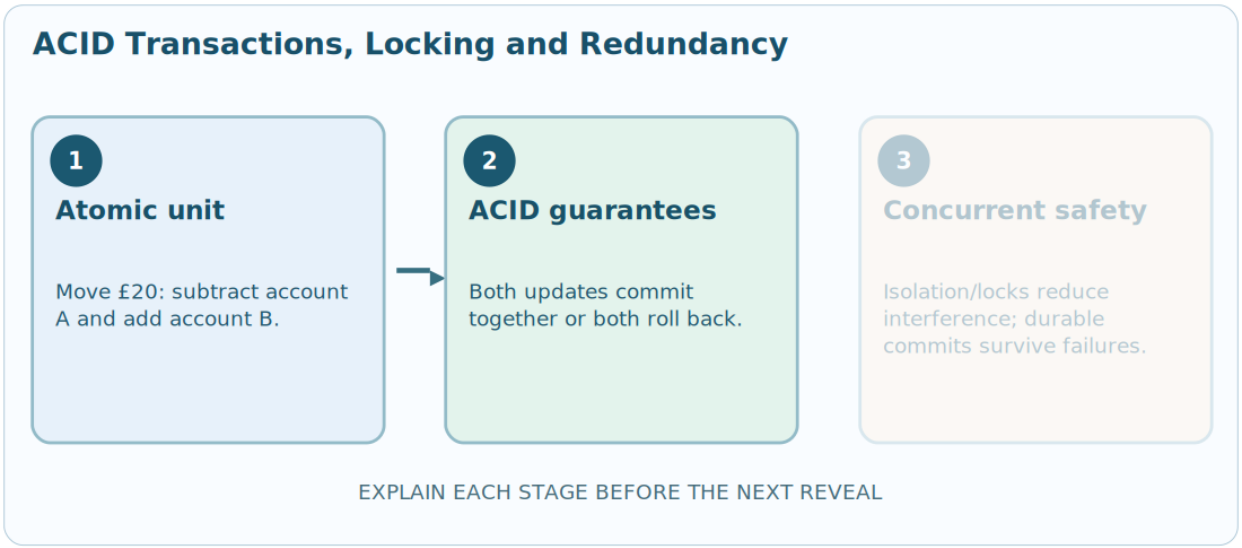
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



ACID Transactions, Locking and Redundancy: Atomic unit, ACID guarantees, Concurrent safety

New Learning

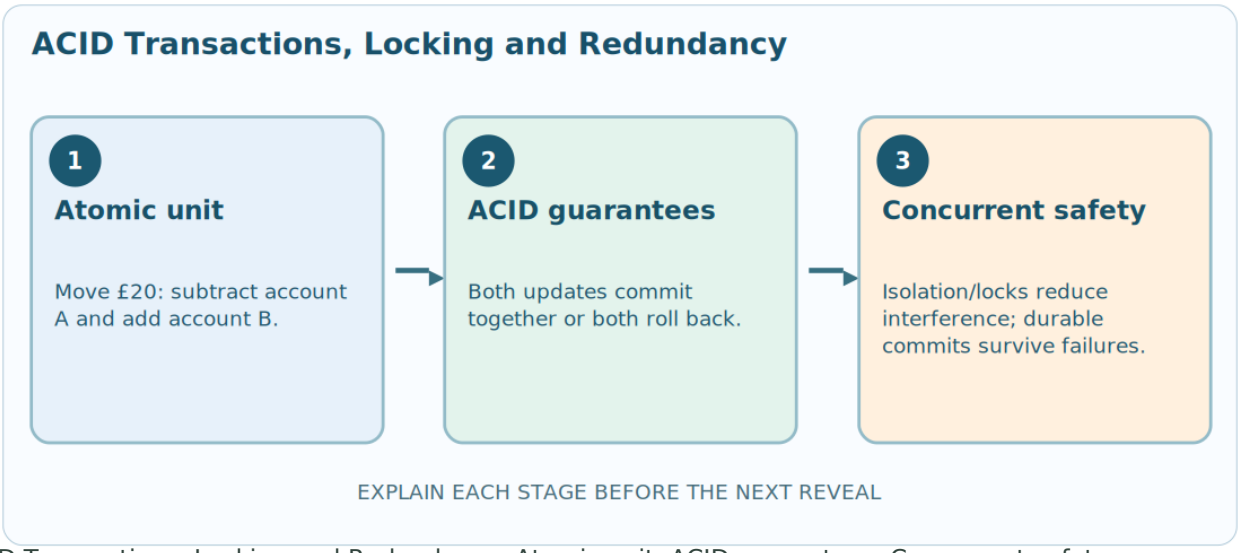


ACID Transactions, Locking and Redundancy: Atomic unit, ACID guarantees, Concurrent safety

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



ACID Transactions, Locking and Redundancy: Atomic unit, ACID guarantees, Concurrent safety

I DO — WORKED EXAMPLE

Follow the worked method

A transfer subtracts £20 from Account A but crashes before adding £20 to Account B. Which ACID property matters most?

Subtract £20 from A

Identify the relevant information and make a prediction.

Crash before adding to B

Explain the change using correct technical terms.

Rollback both changes

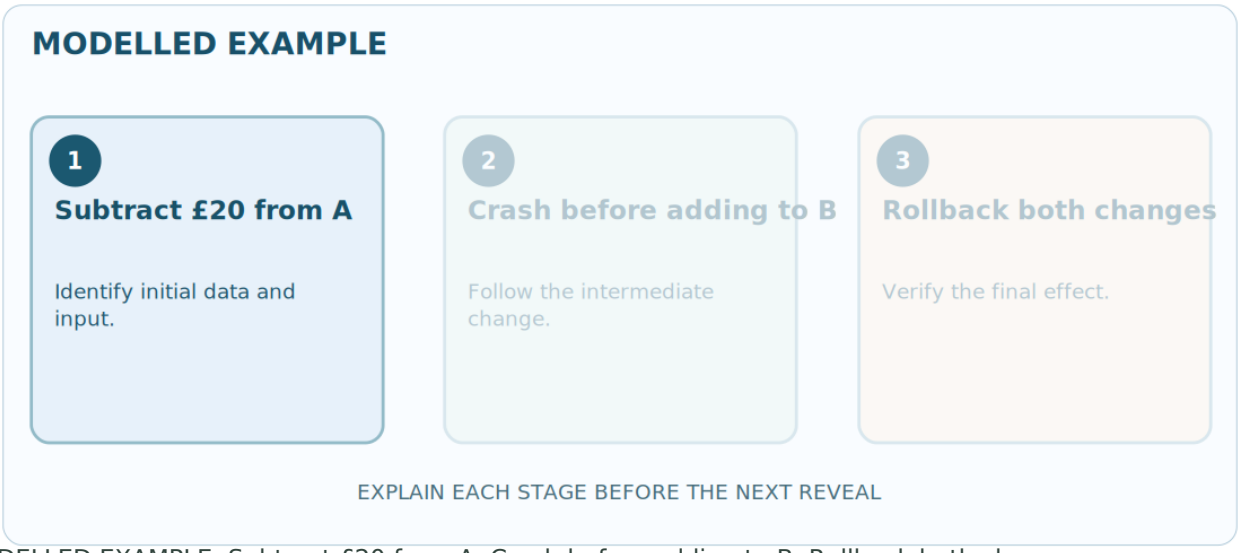
Atomicity means the transfer is all-or-nothing. A DBMS transaction can roll back partial changes. Consistency checks valid constraints; isolation keeps concurrent transfers safe; durability preserves committed transfers.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A transfer subtracts £20 from Account A but crashes before adding £20 to Account B. Which ACID property matter

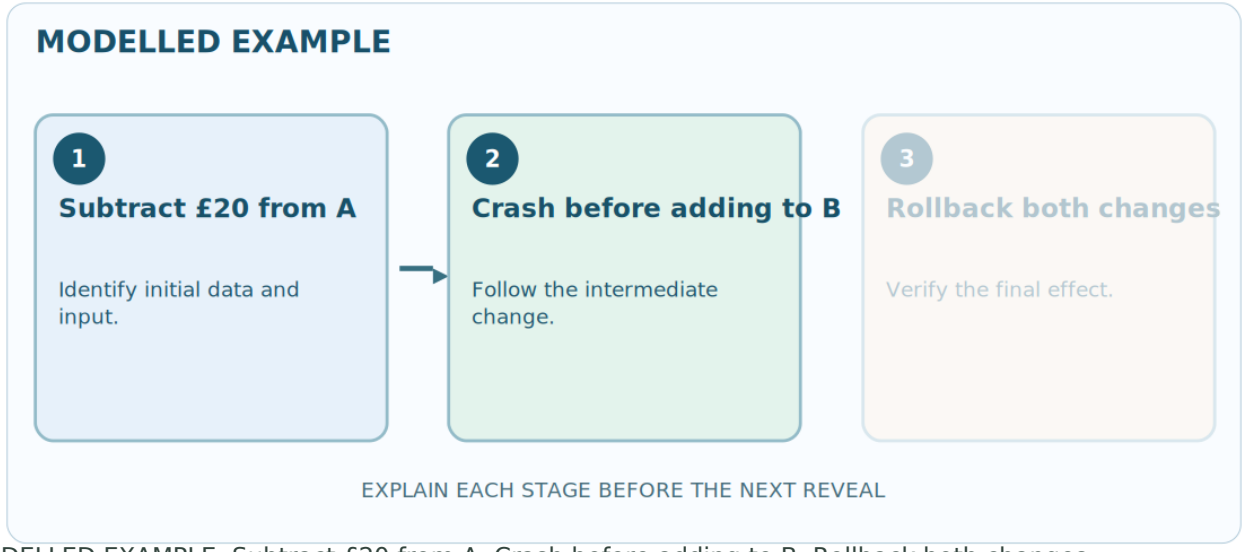
Identify the relevant information and make a prediction.



MODELLED EXAMPLE: Subtract £20 from A, Crash before adding to B, Rollback both changes

A transfer subtracts £20 from Account A but crashes before adding £20 to Account B. Which ACID property matter

Identify the relevant information and make a prediction.



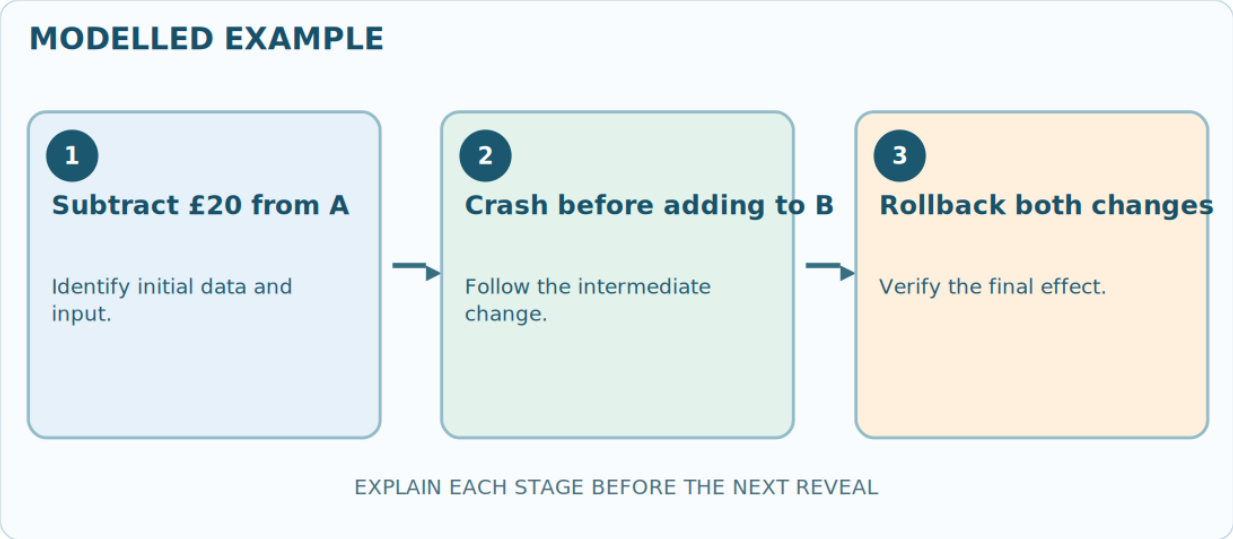
MODELLED EXAMPLE: Subtract £20 from A, Crash before adding to B, Rollback both changes

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

A transfer subtracts £20 from Account A but crashes before adding £20 to Account B. Which ACID property matter

Identify the relevant information and make a prediction.



MODELLED EXAMPLE: Subtract £20 from A, Crash before adding to B, Rollback both changes

WE DO — GUIDED PRACTICE

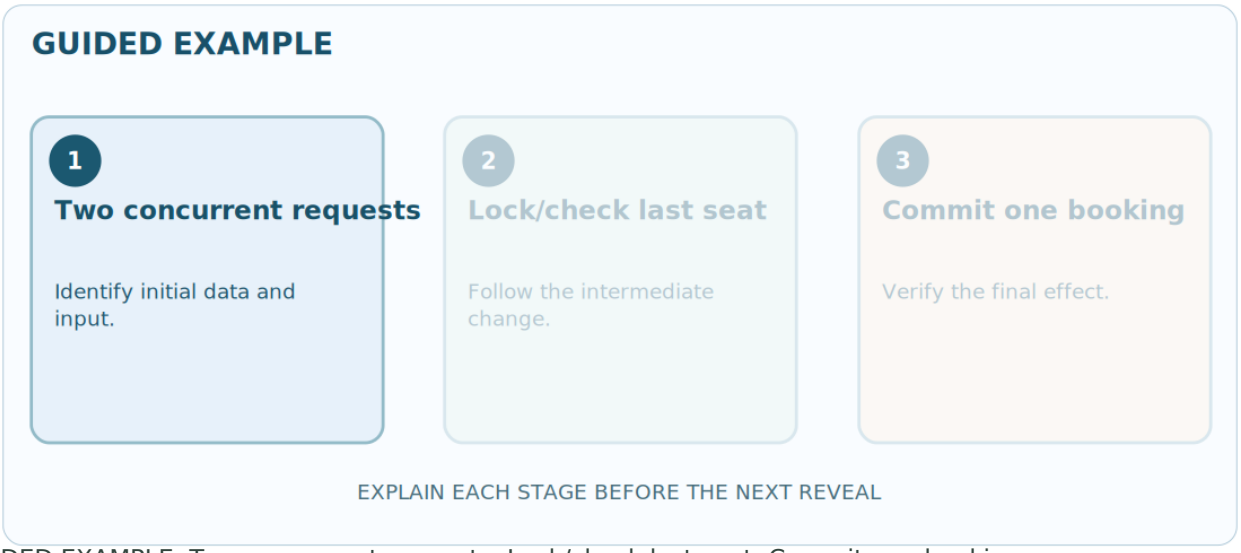
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Two users attempt to reserve the last seat at the same time. How do transactions prevent double booking?

Identify the relevant information and make a prediction.



GUIDED EXAMPLE: Two concurrent requests, Lock/check last seat, Commit one booking

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Two concurrent requests Identify the relevant information and make a prediction.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Expand ACID and explain atomicity and durability. [6]

Question 2

Explain why two concurrent users editing one record can cause a lost update. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Evaluate replication, RAID and backups for database resilience. [8]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

RAID or replication guarantees recovery after an accidental DELETE.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

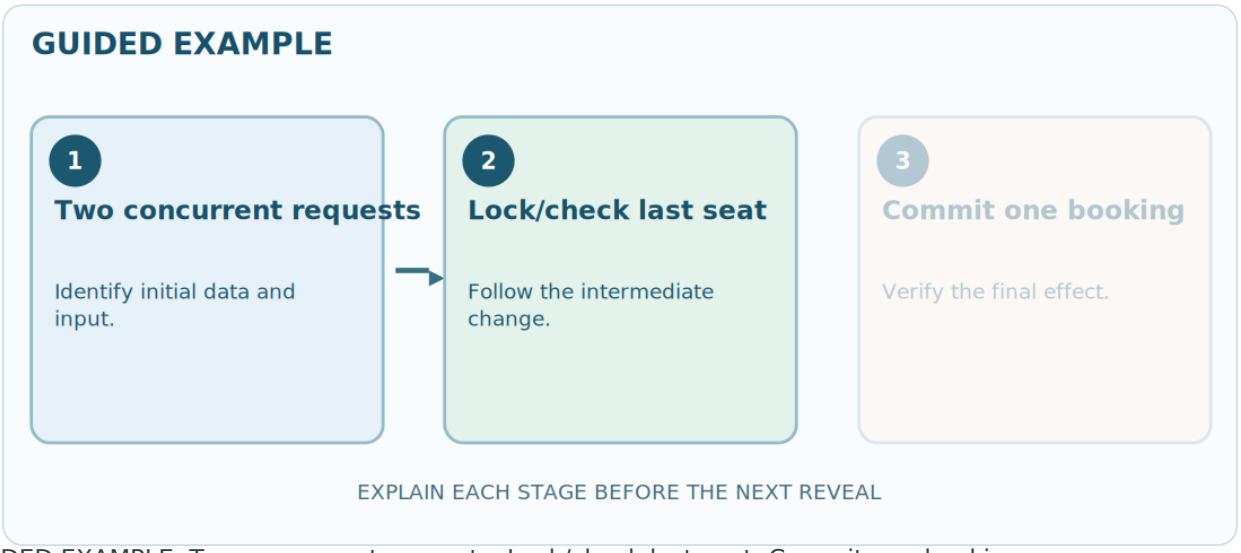
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Two users attempt to reserve the last seat at the same time. How do transactions prevent double booking?

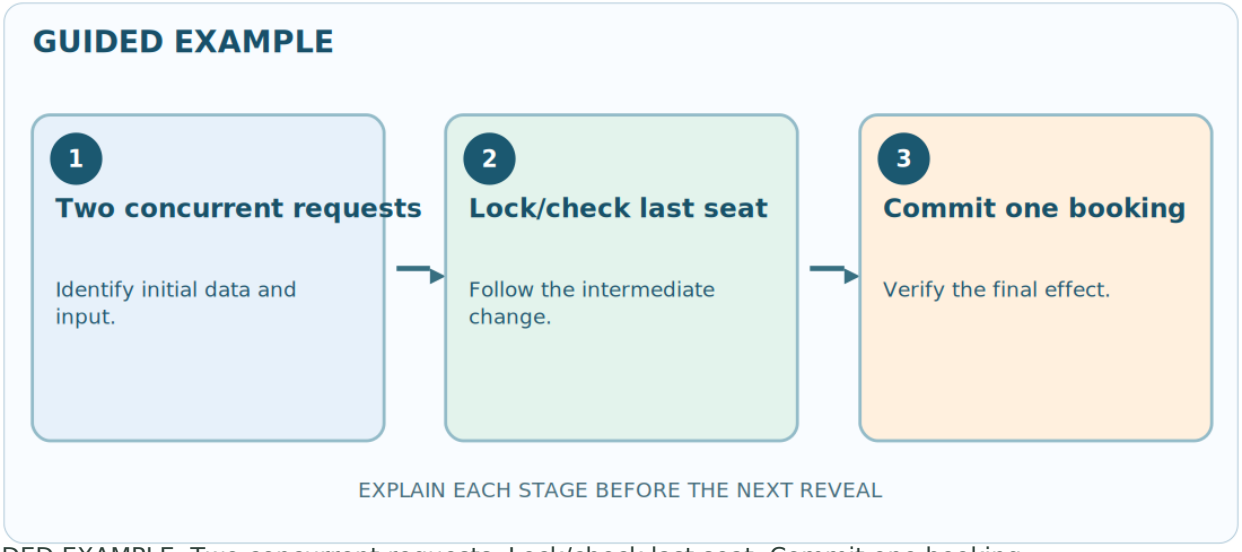
Identify the relevant information and make a prediction.



GUIDED EXAMPLE: Two concurrent requests, Lock/check last seat, Commit one booking

Two users attempt to reserve the last seat at the same time. How do transactions prevent double booking?

Identify the relevant information and make a prediction.



GUIDED EXAMPLE: Two concurrent requests, Lock/check last seat, Commit one booking

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Lock/check last seat

Explain the change using correct technical terms.

Commit one booking

Use an atomic inventory check and reservation update inside an appropriately isolated transaction, with row locking or other concurrency controls. Only one should succeed; second sees no remaining seat.

IMPROVE: Explain one correction to your own reasoning.